

Desafio Dev. Pleno: Full Stack PHP + REACT

1. Objetivo Do Projeto

O objetivo deste desafio é desenvolver um Gerenciador de Empréstimos de Livros simplificado. O sistema permitirá que usuários autenticados visualizem um acervo e solicitem o empréstimo de exemplares disponíveis, com controle de estoque e histórico.

O desafio deve demonstrar maturidade em **consistência de dados, concorrência, qualidade de API (retornos corretos), SQL, Idempotência (quando usar), e frontend com gerenciamento de estado**. Esses detalhes devem ser identificados em quais regras e casos de uso devem ser implementados.

2. Requisitos Técnicos E Ferramentas

- **Linguagem Backend:** PHP 8.3 ou superior.
- **Framework Backend:** Laravel 12.
- **Linguagem Frontend:** React com TypeScript.
- **Comunicação:** fetch ou SWR nas APIs.
- **Interface:** Material UI (MUI) para componentes de tabela e botões.
- **Banco de Dados:** MySQL.
- **Ambiente:** Docker funcional (docker compose).

3. Escopo De Desenvolvimento

3.1. Autenticação

- Implementar o sistema de login e registro nativo do Laravel Breeze.
- Rotas protegidas no backend e frontend.

3.2. Modelagem de Dados O banco de dados deve conter, no mínimo, as seguintes estruturas:

- Atributos para título, autor, ISBN e quantidade de cópias disponíveis.
- Registro de empréstimos contendo a relação entre o usuário e o livro, além da data da transação.
- Criar um Database Seeder para inserir ao menos 10 livros no sistema para teste.

3.3. Regras de Negócio e Casos de Uso (sem necessidade de desenvolver a ação em tela para todos os casos de uso)

- Exibir todos os livros cadastrados.
- Cadastrar um livro novo.
- Usuário deve conseguir fazer empréstimo de um ou mais livros.
- Um livro com zero cópias disponíveis não pode ser emprestado.
- Limite de empréstimos ativos por usuário deve ser 3.
- Usuário deve devolver um livro.
- Após 7 dias o empréstimo deve ser considerado atrasado se não devolvido.

3.4. Frontend e UI

- A listagem de livros deve ser obrigatoriamente implementada utilizando o componente Table do Material UI.
- A interface deve ser responsiva.
- A interface deve se comportar como um SPA.
- Usar cache de resposta quando necessário.

3.4. Ambiente

- Configurar o ambiente Docker de forma que o projeto seja iniciado com comandos padrão (ex: docker compose up).

3.4. Testes

- Teste de concorrência no empréstimo de livros, dois usuários emprestando o mesmo livro.
- Teste da regra de limite de empréstimos por usuário.
- Teste e2e conferindo a listagem de livros no frontend.

4. Requisitos Obrigatórios De Implementação

- **Consultas SQL:** A tela de "Meus Empréstimos" deve listar os livros alugados pelo usuário atual. Esta consulta específica deve ser realizada utilizando SQL Raw (DB::select ou DB::raw), executando um JOIN entre as tabelas de livros e empréstimos.
- **Arquitetura:** O uso de Action Pattern para a lógica será considerado um diferencial importante na avaliação de organização de código.

5. Orientações Sobre Integridade e IA

Este desafio visa avaliar a capacidade analítica, organização de código e conhecimento de sintaxe do candidato. Não é permitido o uso de inteligência artificial para a geração integral do projeto. O uso de ferramentas de auxílio é aceitável para consultas pontuais, porém o candidato deve estar apto a explicar toda a arquitetura e lógica implementada durante a entrevista técnica.

6. CRITÉRIOS DE AVALIAÇÃO

- Capacidade de configurar e rodar o ambiente via Docker.
- Domínio da sintaxe PHP e recursos do Laravel.
- Qualidade da consulta SQL manual e tratamento de joins.
- Uso da biblioteca Material UI e o ecossistema React.
- Separação de responsabilidades (Controllers e Actions).
- Integridade e concorrência
- Qualidade de API (erros, validação, paginação)

7. INSTRUÇÕES DE ENTREGA

1. Disponibilizar o código-fonte em um repositório privado (GitHub ou GitLab), compartilhar com francis@besistemas.com.br.
2. Incluir um arquivo README.md com as instruções de instalação e execução do projeto.